Batucan, Zly Hanson A.

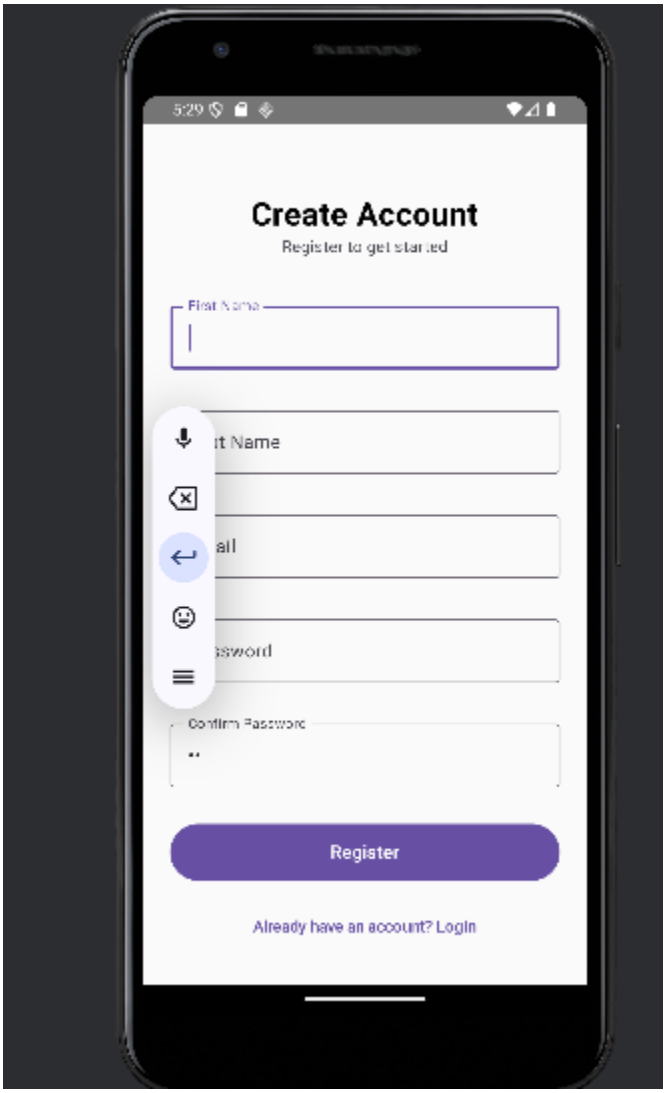
IT342

StockEase

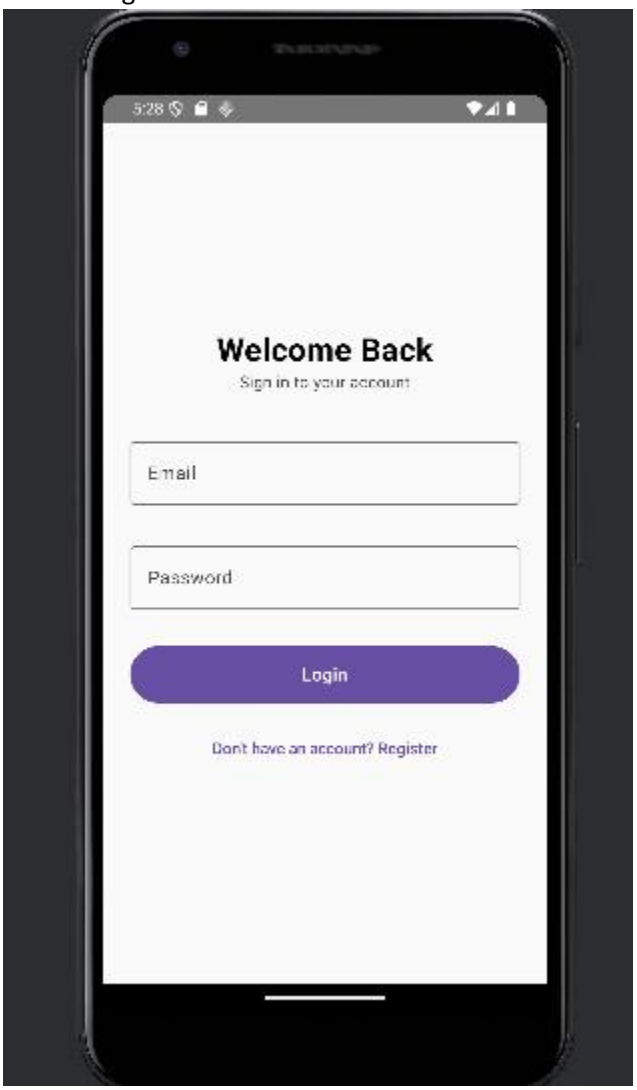
Phase 2:

GitHub Link:
https://github.com/spong4/IT342-Batucan-StockEase

Mobile Registration



Mobile Login



How Registration Works

1. **User fills out the form** — First Name, Last Name, Email, Password, Confirm Password on `RegisterScreen.kt`
2. **Input validation runs first** (before any API call):
   - First name and last name cannot be blank
   - Email must be a valid format (using `android.util.Patterns.EMAIL_ADDRESS`)
   - Password must be at least 8 characters
   - Confirm password must match password
   - If any validation fails, error messages appear **below each field** without touching the API
3. **API call is made** if validation passes:
   - A `RegisterRequest` object is created with the form data

    - Retrofit sends a **POST** request to `/api/v1/auth/register`
    - The request runs inside a **coroutine** (`scope.launch`) so it doesn't freeze the UI
4. **Response is handled:**
    - If `response.success == true` → shows success message and navigates to Login screen
    - If `response.success == false` → shows the error message from the backend (e.g. "Email already exists")
    - If the server is unreachable → shows "Could not connect to server"

How Login Works

1. **User fills out the form** — Email and Password on `LoginScreen.kt`
2. **Input validation runs first:**
    - Email cannot be blank and must be valid format
    - Password cannot be blank and must be at least 8 characters
3. **API call is made** if validation passes:
    - A `LoginRequest` object is created with email and password
    - Retrofit sends a **POST** request to `/api/v1/auth/login`
    - Also runs inside a coroutine
4. **Response is handled:**
    - If `response.success == true` → navigates to `HomeScreen`
    - If `response.success == false` → shows error message (e.g. "Invalid credentials" from `AUTH-001`)
    - If server is unreachable → shows "Could not connect to server"
5. **Navigation after success:**
    - Uses `popUpTo("login") { inclusive = true }` so the user **cannot go back** to the login screen by pressing the back button

---

API Integration

The API integration is built using **Retrofit** and follows this structure:

`RetrofitClient.kt` — sets up the connection:

- Defines the `BASE_URL` pointing to your backend (`http://10.0.2.2:8080/` for emulator)
- Uses `GsonConverterFactory` to automatically convert JSON responses into Kotlin data classes
- Is an `object` (singleton) so only one instance exists throughout the app

`ApiService.kt` — defines the endpoints:

- `@POST("api/v1/auth/register")` → maps to your Spring Boot register endpoint
- `@POST("api/v1/auth/login")` → maps to your Spring Boot login endpoint
- Both are `suspend` functions meaning they run asynchronously